光靠 Prompt 守不住架構

用 Lint 與 Architecture Test 打造 Agentic Coding 防護牆

> **提醒會被忘記，檢查不會。**

Neo Hsu

02 · 破題

你是不是也遇過這些問題？

Q1

為什麼產生的 Code，總是不符合 coding style？

Q2

為什麼 AI 好像不懂我的 module 在做什麼？

Q3 · 今天的重點

為什麼產生的 Code，和我的 codebase 架構不一樣？

Q4

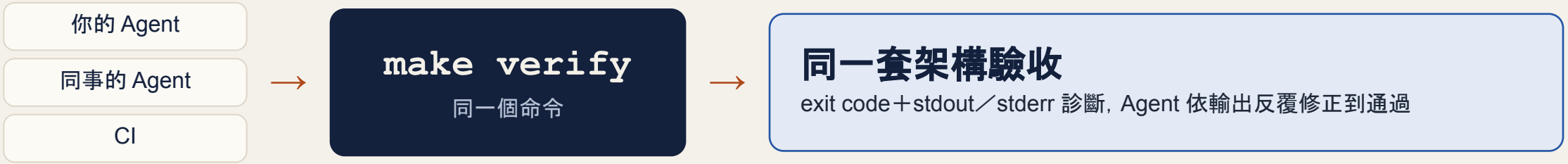
為什麼每次發 PR，總是沒照我給的 template？

這些規範都寫過了：寫在 Prompt、寫在範本、寫在 Review 留言裡，但有時候會沒有執行。

四個問題，同一個根因：都在靠提醒。

03 · 核心主張

換哪個 coding agent，都收斂到同一套架構



不依賴

- Prompt 的措辭與長度
- 特定 Agent 的 hook、skill 或外掛
- 模型這一次有沒有照做

只依賴

- 專案內的 Lint／Test 與共用命令
- 工具輸出的 exit code 與診斷
- 每個 Agent 都會的事：執行命令、讀輸出

Prompt 只需要一句「改完請跑 make verify」；漏跑了，**Git hook 與 CI 也會用同一個命令擋下** 。

Agent 可以不同，驗收必須相同。

需求是否正確（Graph 分析、SDD、BDD）是另一個主題；本場假設需求已對齊。

04 · 路線圖

分四個階段，把提醒變成工具輸出

階段 1

Style

Q1 不符合 coding style

formatter 統一格式；linter 規則附上修正訊息

→

階段 2

Module 意圖

Q2 不懂 module 在做什麼

責任與理由寫進文件；AGENTS.md 指向入口

→

階段 3 · 重點

架構邊界

Q3 和 codebase 架構不同

depguard、路徑檢查、能指出修正入口的診斷

→

階段 4

PR 流程

Q4 PR 沒照 template

CI 檢查 PR 描述；保護設定決定能否合併

每個階段都接同一個修正迴圈：執行檢查 → 讀工具輸出 → Agent 修正 → 再驗。

從最容易判定的開始，一層一層往上加。

為什麼不符合 coding style？

格式：交給 formatter

gofmt 能自動統一的，不必在對話裡提醒。

慣例：交給 linter，附上原因

msg 會出現在違規訊息裡，Agent 讀了就知道怎麼改。

.golangci.yml 節錄 · forbidigo

```
forbidigo:
  forbid:
    - pattern: '^fmt\.Print(f|ln)?$'
      msg: Write through command I/O or
      an injected logger instead of …
    - pattern: '^http\.DefaultClient$'
      msg: Construct an explicit
      http.Client with transport and …
```

不只禁止，還說明該改成什麼。

只把**有共識、可明確判定**的規範寫成規則；還有爭議的風格，先在團隊談好。

> **能判定的風格，就不該出現在 Review 留言裡。**

06 · 階段 2 | Module 意圖

為什麼 AI 好像不懂我的 module 在做什麼？

只說模式名稱

「我們採用 Clean Architecture」

Agent 只看得到程式碼，看不到責任與理由。

說清楚責任

明確列出模塊區分和用途

資料夾

提供地圖

架構文件與設計決策

解釋責任與理由

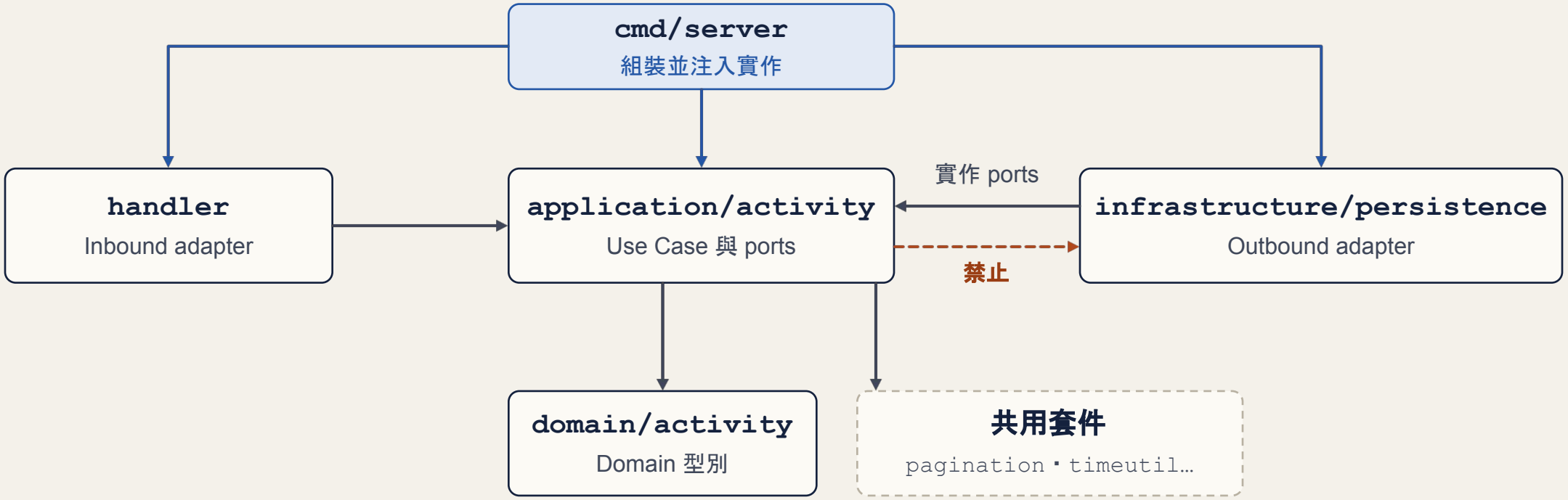
程式與測試

提供可檢視的例子

AGENTS.md／CLAUDE.md 只要指向這些入口；這一階段，Prompt 和文件仍然重要。

> **Agent 讀得到程式碼，讀不到你沒寫下的理由。**

給 Agent 一張地圖: Activity 的依賴方向



階段 3 都用這個例子 箭頭＝原始碼 import 方向，不是執行時呼叫。application 與 handler 都不得 import infrastructure。

看懂這條橘線，就看懂階段 3 的每個違規。

08 · 階段 3 | 架構邊界

為什麼產生的 Code，和 codebase 架構不一樣？

階段 2 讓 Agent 看懂了，但看懂不等於做到：要把意圖寫成可判定的規則。

意圖·寫在文件

application/activity 協調 Activity 的操作，不負責資料存取細節。

理由：應用流程不需要知道資料庫驅動與連線。

→

規則·可判定

Application 不得有 import 路徑通往 Infrastructure。

直接與間接都算；箭頭是原始碼 import，不是執行時呼叫。

→

樣本·驗證規則本身

合法　直接違規　間接違規

例外若存在，要有明確範圍、原因與退場條件。

先寫規格，再選工具： Lint、依賴分析器、自訂測試，都只是實作方式。

AGENTS.md／CLAUDE.md 只要指向架構文件、程式與測試這些入口，不必複製全文。

規則要能判定對錯，才交得給工具。

第一道：depguard 擋直接依賴

.golangci.yml 節錄（module 路徑縮寫為 <module>）

```
depguard:
  rules:
    application-production:
      list-mode: strict
      files:
        - "${base-path}/internal/application/**/*.go"
      allow:
        - $gostd
        - <module>/internal/domain
        - <module>/internal/pagination
    handler-production:
      deny:
        - pkg: <module>/internal/infrastructure
          desc: handlers must depend on application ports, …
```

`list-mode: strict`
白名單：沒列的都擋。Application 直接 import persistence，不在清單就擋下。

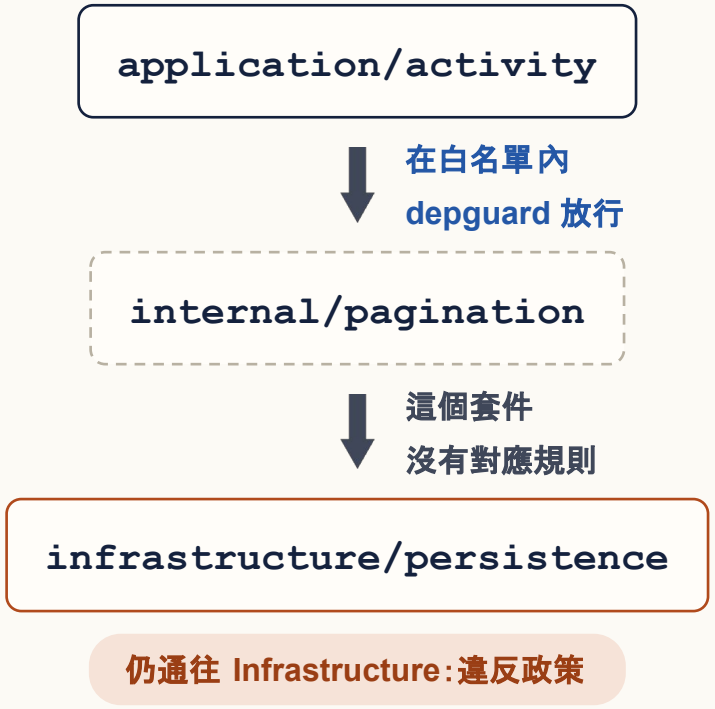
`desc:`
deny 規則的 desc 會出現在違規訊息裡，就是修正提示；白名單只會說「不在清單」。

`nolintlint: require-explanation`
用 //nolint 關規則，必須寫明原因並指定 linter，不能默默放寬。

只看直接 import
只檢查指定檔案的直接依賴，不宣稱已覆蓋整條路徑。

違規訊息寫得好，就不必再補 Prompt。

白名單的縫：藏進共用套件的間接路徑



depguard 一次只看一條邊

每一段單獨看都合法，串起來卻通往 Infrastructure。

不是 Agent 故意繞

它可能以為「放進允許的套件就可以」；該補的是檢查。

修缺口，不寫黑名單

讓檢查跟上整條路徑的政策，不追著套件名字補規則。

每一段都合法，不代表整條路合法。

示意情境

Architecture Test：檢查路徑，不只看單一邊

自訂 Go test · go/packages 載入 package import graph

載入 import graph → 從 Application 出發 → 遍歷，不重複訪問 → 找到 Infrastructure

輸出路徑 `application/activity → pagination → infrastructure/persistence`

檢查器本身也要驗證

合法樣本　違規樣本　載入失敗　掃不到預期套件　空掃描

「什麼都沒檢查到」不等於沒有違規。

明確宣告掃描範圍

module　GOOS　build tags　解析範圍　generated code　test packages

一次執行，只代表選定的組態。

> 政策管的是整條路，檢查也要看整條路。

12 · 階段 3 | 診斷回饋

診斷不只報錯，也指出既有修正入口

`Architecture violation` 只有這行：Agent 還是要猜哪裡錯、為什麼、這個專案期待怎麼修。

示意格式：自行設計，不是 depguard 或 dependency-cruiser 的預設輸出

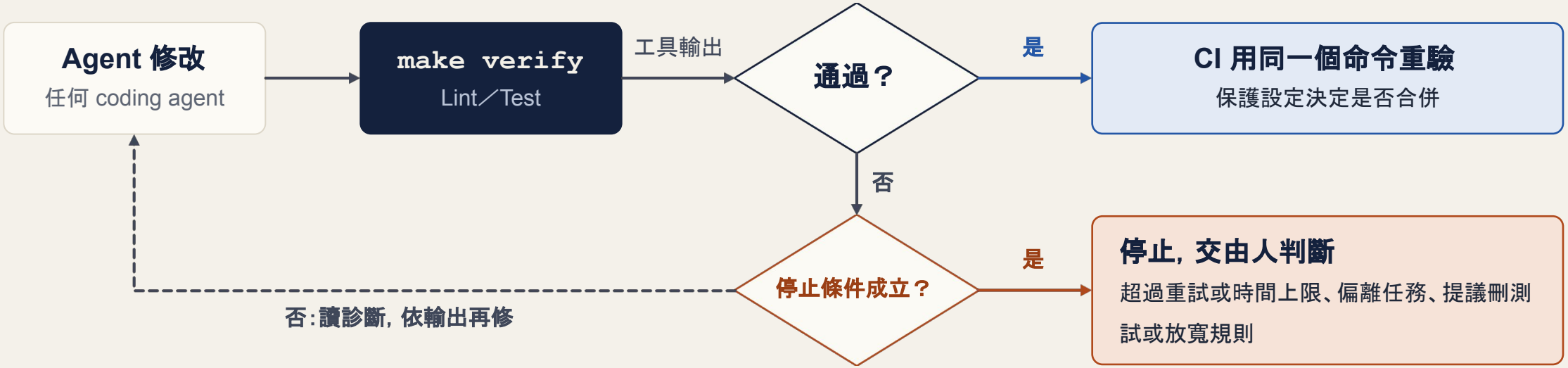
規則	Application 不得有 import 路徑通往 Infrastructure
路徑	`application/activity → pagination → infrastructure/persistence`
理由	應用流程不需要知道資料庫驅動與連線細節
修正入口	改用 Activity 自己的 port，由 cmd/server 注入 persistence 實作
重驗命令	`make verify`

⚠ 診斷裡的路徑也要維護：入口不存在，就會變成另一份過期文件。

> **好的錯誤訊息，就是下一輪的 Prompt。**

13 · 修正迴圈

把回饋接成有限的修正迴圈



⚠ **回傳要完整：**非零 exit code 不能被吃掉，stderr 與診斷要保留；沒有輸出不等於成功。

減少的是逐輪人工提醒，**不是移除意圖、權限與人工責任** 。

> **修不過就停下來，不能自己降低標準。**

為什麼每次發 PR，總是沒照 template 走？

範本只是預填文字

沒有任何東西驗證它有沒有被照著填。

Agent 常用 CLI 或 API 建 PR

網頁上的範本自動帶入，不一定會發生。

Git hook 管不到

PR 描述不在 git 裡，這一關只能放在 CI。

示意：CI 檢查 PR 描述

```
$ make check-pr
FAIL PR description
缺少段落：「測試方式」「影響範圍」
請依 .github/pull_request_template.md 補齊
exit 1
```

建立／更新 PR → CI 檢查段落 → Agent 依輸出補齊

設成 required check 後，沒補齊就不能合併。

範本是提醒，CI 檢查才是驗收。

15 · 階段 4 | 三層防線

三層防線：Agent 自己跑、Git hook、CI

1 · 主動

Agent 自己跑

- 同一個入口：make verify
- 工具版本、環境、掃描範圍要明確
- 回饋最快，直接讀輸出修正

2 · 自動

Git hook

- commit／push 時自動執行
- 可用 --no-verify 繞過，不算強制
- 用 core.hooksPath 或 lefthook 安裝

3 · 強制

CI＋保護設定

- 對待合併版本重驗
- Required checks、必要 Review、bypass 權限
- skipped 或 neutral 不等於成功

⚠ 規則、測試、CI 與豁免清單的修改要另行審查；CODEOWNERS 要搭配強制生效的保護設定。

Hook 負責攔下，CI 負責把關。

16 · 收尾

把反覆提醒，變成可執行的回饋

Prompt
解釋設計意圖

Lint／Test
驗證具體約束

診斷
讓任何 Agent 都能依輸出修正

人
判斷需求與設計取捨

從一條有共識、可驗證的規則開始。

> **提醒會被忘記，檢 查不會。**

Q & A

Thanks

備用頁

問答時可能用到

Prompt 與驗收

Activity 三個症狀

修正後的依賴圖

設計契約測試

Git hook 細節

備用 · Activity 情境

每個人用不同 Agent，架構卻各自走樣

情境｜三位同事各用習慣的 coding agent，在 Activity 加新功能，功能測試都通過

01

誰依賴誰

Application 直接 import persistence 實作

02

誰負責組裝

Use Case 不用注入的 port，自己建立具體實作

03

包裝真的修好了嗎？

提醒之後，資料存取被搬進共用的 pagination 套件

Review 一直在 **重複提醒同樣的事** 。

換一個人、換一個 Agent，又要從頭再提醒一次。

> **同樣的提醒說到第三次，就該變成檢查。**

讀過規範 ≠ 這次修改符合規範

Prompt／文件

解釋設計意圖與既有入口

說明為什麼，以及往哪裡找。

＋

可執行檢查

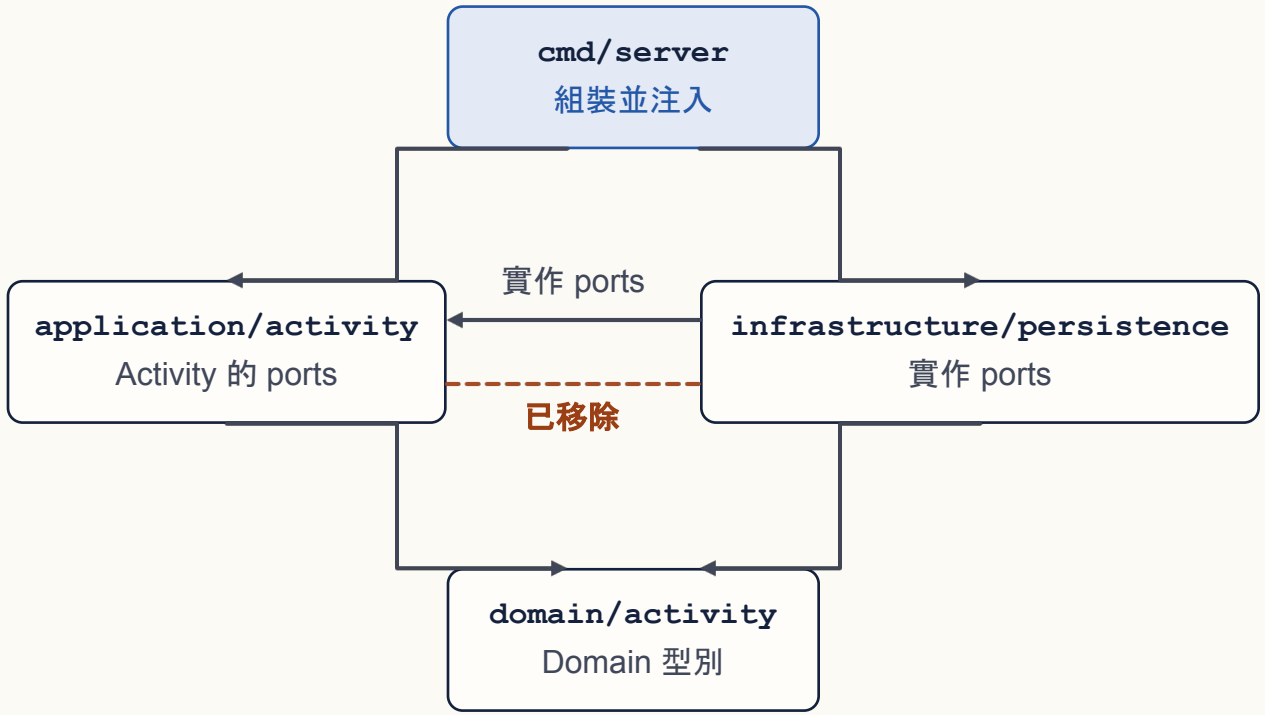
取得這次修改的具體證據

減少對反覆提醒與模型自述的依賴。

不是用 Test 取代 Prompt，而是補上可執行的驗收。

備用 · 修正後的依賴圖

真正修好：回到既有設計



驗收

- ✓ 禁止路徑消失
- ✓ 既有回歸測試仍通過

箭頭是原始碼 import，不是 runtime 呼叫；執行時 Use Case 透過 port 呼叫 persistence 實作。

Go 的實作不需顯式宣告 interface；persistence import Activity 的 ports 來實作，方向由外往內，不形成 cycle。

依賴圖漂亮，還不等於契約一致。

備用 · 設計契約

設計契約，也要有可執行的例子

方法簽名一致，不代表行為一致。

共用契約案例 · 教學假設

找不到指定的 Activity

對外提供 Domain 定義的錯誤，不洩漏資料庫驅動的錯誤

成功儲存後再讀取

取回約定的欄位

→

測試替身

跑同一組案例；只測它，不能宣稱資料庫實作也正確。

PostgreSQL Adapter

需要受控資料庫、資料準備與清理。

Lint／依賴檢查 守結構

契約測試 守選定的互動語意

通過只代表這組案例的證據，不是完整證明所有可替換性、交易或併發語意。

備用 · Git hook

Git hook：不靠 Agent 記得跑

1 · 主動
Agent 自己跑
make verify，回饋最快

→

2 · 自動
Git hook
commit／push 時觸發，漏跑也擋下

→

3 · 強制
CI＋保護設定
對待合併版本重驗，決定能否合併

示意：Agent 執行 git commit

```
$ git commit -m "cancel order"
pre-commit: make verify
FAIL Application → Infrastructure
application/activity → pagination
→ infrastructure/persistence
exit 1, commit aborted
```

hook 在 git 層，哪個 Agent commit 都會觸發；失敗輸出直接回到 Agent，接上同一個修正迴圈。

Hook 不是強制

- 可用 --no-verify 繞過；Agent 提議這麼做，就是停止條件
- .git/hooks 不進版控，要用 core.hooksPath 或 lefthook 等工具安裝
- pre-commit 只跑快的檢查，契約測試留給 pre-push 或 CI